

FreshBasket Cloud Infrastructure

AWS Architecture Design and Implementation Report

Student ID: 14594648

Student Name: Thi Minh Thu Nguyen

Subject: Cloud Computing and SaaS (42904)

Assignment 3 AWS Infrastructure Design and Deployment

Table of Contents

1.1 Architecture Diagram	3
Traffic Flow.....	4
1.2 Component Justification.....	4
1.3 Scalability, Disaster Recovery, High Availability, and Fault Tolerance	7
Scalability	7
Disaster Recovery	7
High Availability	7
Fault Tolerance	8
1.4 Assumptions	8
1.5 Complete List of AWS Services Used	8
Screenshot 1 - Custom VPC.....	9
Screenshot 2 - EC2 Security Group	10
Screenshot 3 - RDS Security Group.....	10
Screenshot 4 - Custom key pair	11
Screenshot 5 - Custom AMI.....	11
Screenshot 6 - RDS Instance Creation Error	12
Screenshot 7 - Elastic Beanstalk Environment Dashboard	12
Screenshot 8 - Auto Scaling Configuration.....	13
Screenshot 9 - Application Running End-to-End.....	14
Screenshot 10 - Email Notifications.....	15
References.....	16

PART 1 - ARCHITECTURE DOCUMENT

1.1 Architecture Diagram

The diagram below shows how all the AWS components connect together for the FreshBasket app. When a user makes a request, it comes in through the Internet Gateway, hits the Load Balancer, and then gets sent to one of the EC2 instances running the app. Those EC2 instances are spread across two Availability Zones inside a custom VPC. The app then reads from or writes to the RDS MySQL database. CloudWatch watches the CPU and triggers scaling when needed, and SNS handles email alerts.

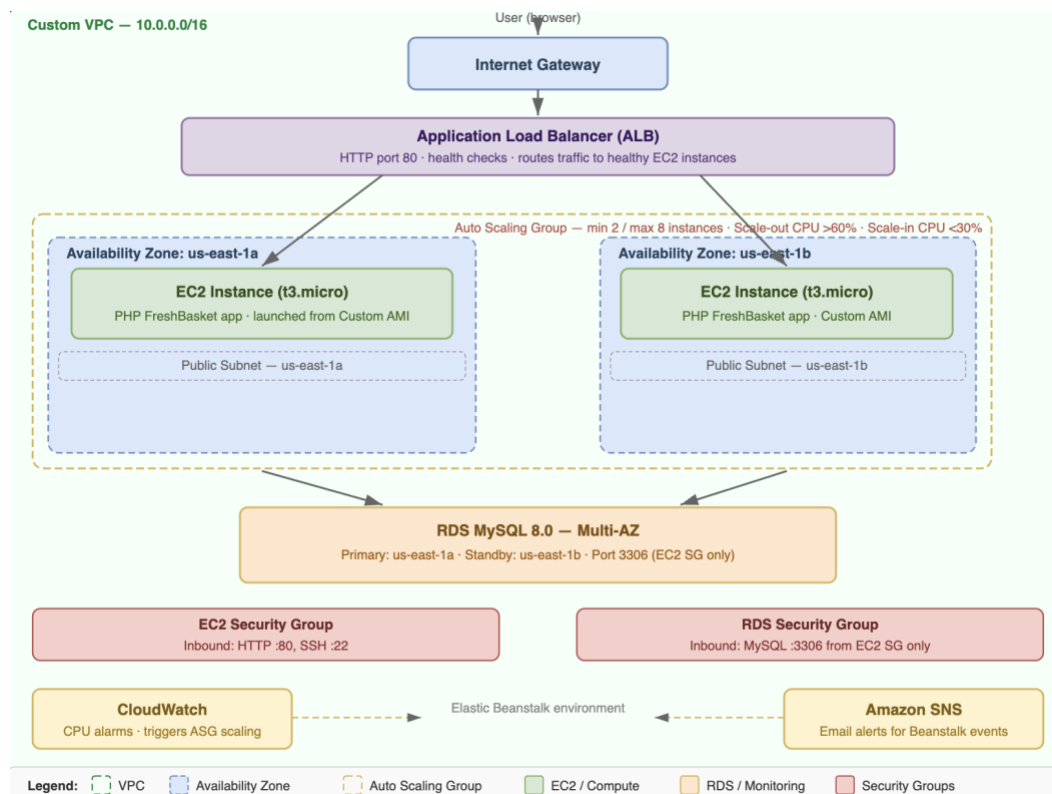


Figure 1.1 FreshBasket AWS Architecture (Diagram was created using draw.io (app.diagrams.net))

Traffic Flow

1. A user opens the app in their browser and submits an order. The HTTP request comes in on port 80 and passes through the Internet Gateway into the VPC.
2. The Application Load Balancer picks up the request and forwards it to one of the healthy EC2 instances either in us-east-1a or us-east-1b.
3. The EC2 instance runs the PHP app and handles the request. If the user is submitting an order, it runs an INSERT to save it. If the page is loading, it runs a SELECT to pull existing orders from the database.
4. The RDS MySQL database returns the result, the EC2 instance builds the HTML page, and the response goes back to the user through the Load Balancer.
5. CloudWatch is watching the average CPU across all instances in the Auto Scaling Group. If it goes above 60%, the ASG launches more EC2 instances from the Custom AMI. If it drops below 30%, it removes instances but always keeps at least 2 running.
6. Whenever there is a health change or a scaling event in Elastic Beanstalk, SNS sends an email notification to the nominated address.

1.2 Component Justification

The table below lists each AWS service used and explains what it does and how it supports scalability, high availability, disaster recovery and fault tolerance.

AWS Service	Role in FreshBasket	Scalability	High Availability / DR / Fault Tolerance
Elastic Beanstalk	Handles the whole deployment creates the EC2 instances, load balancer, Auto Scaling Group and CloudWatch alarms automatically	Scales the environment up or down based on the ASG rules	Monitors instance health and replaces unhealthy ones without manual intervention

AWS Service	Role in FreshBasket	Scalability	High Availability / DR / Fault Tolerance
EC2 (t3.micro)	Runs the PHP FreshBasket app. Multiple instances run at the same time across two AZs	The ASG adds more EC2 instances when CPU gets high and removes them when it drops	If one instance goes down, traffic just goes to the remaining instances
Custom AMI	A pre-built image with PHP and the app already installed so new instances start up the same way every time	Speeds up scale-out because there is no setup needed on launch	Makes sure all instances are identical - reduces the chance of config issues
Application Load Balancer (ALB)	Sits in front of the EC2 instances and routes incoming traffic to healthy ones	Handles more requests by spreading load across all running instances	Does health checks and stops sending traffic to any instance that fails
Auto Scaling Group (ASG)	Keeps between 2 and 8 EC2 instances running. Scales out when CPU > 60%, scales in when CPU < 30%	Automatically adjusts the number of instances based on actual demand	Always keeps at least 2 instances running so there is no single point of failure
Custom VPC	A private network that contains all the AWS resources for this project	Has subnets in two AZs so resources can be spread across them	If one AZ goes down, resources in the other AZ keep running

AWS Service	Role in FreshBasket	Scalability	High Availability / DR / Fault Tolerance
Public Subnets (×2)	One subnet in us-east-1a, one in us-east-1b - EC2 instances are placed in both	Distributing instances across two subnets spreads the workload	If one AZ has an outage, the other subnet and its instances are still available
Internet Gateway	Connects the VPC to the internet so users can reach the app	Scales automatically - no configuration needed	Managed by AWS so it is highly reliable
RDS MySQL Multi-AZ	Stores all the order data submitted through the FreshBasket form	Storage can scale automatically, read replicas can be added later if needed	Keeps a standby copy in a second AZ. If the primary fails, AWS automatically switches over to the standby within a couple of minutes
Security Groups (EC2 + RDS)	EC2 SG allows HTTP on port 80 and SSH on port 22. RDS SG only allows MySQL on port 3306 from the EC2 security group	Rules apply automatically to all instances in the group	The database cannot be accessed directly from the internet - only EC2 instances can connect to it
CloudWatch	Monitors CPU usage across the Auto Scaling Group and triggers scaling alarms	Scale-out and scale-in alarms fire automatically based on the CPU thresholds	Detects when something is wrong and can trigger a response
SNS	Sends email notifications when	Works independently of	Keeps the team informed of any environment changes or failures

AWS Service	Role in FreshBasket	Scalability	High Availability / DR / Fault Tolerance
	Elastic Beanstalk events happen	the app so it is not affected by app issues	

1.3 Scalability, Disaster Recovery, High Availability, and Fault Tolerance

Scalability

The Auto Scaling Group is set to run between 2 and 8 EC2 instances depending on load. When the average CPU goes above 60%, the ASG automatically starts new instances. When it drops back below 30%, it scales back down. Because all instances launch from the same Custom AMI (which already has PHP and the app installed), they are ready to serve traffic quickly. The Load Balancer spreads requests across however many instances are running at the time, so no single instance gets overloaded. On the database side, RDS storage can scale automatically as well, and read replicas could be added later if needed.

Disaster Recovery

The main concern for disaster recovery is the database if that goes down and data is lost, the whole app breaks. To handle this, RDS MySQL was set up with Multi-AZ enabled. This means AWS keeps a second copy of the database running in us-east-1b at all times. If the primary database in us-east-1a has a problem, AWS automatically switches over to the standby copy within a couple of minutes, and no data is lost because it is kept in sync. For the EC2 instances, since they just run the app code (no data is stored on them), the ASG will just launch new ones from the AMI if any go down.

High Availability

To avoid having a single point of failure, the architecture spreads everything across two Availability Zones. The custom VPC has one public subnet in us-east-1a and one in us-east-1b, and Elastic Beanstalk puts EC2 instances in both. The Load Balancer sits in front and checks that each instance is healthy before sending traffic to it. If an instance fails a health check, the ALB stops routing to it and the ASG replaces it. The RDS Multi-AZ setup also covers the database layer. So even if one entire Availability Zone went down, the app would keep running on the other one.

Fault Tolerance

Two separate security groups are used to control access. The EC2 security group only allows HTTP on port 80 and SSH on port 22, nothing else. The RDS security group only allows MySQL traffic on port 3306, and the source is locked to the EC2 security group ID, meaning only the app instances can connect to the database. Nobody can reach the database directly from outside. CloudWatch monitors the environment and triggers alarms if something goes wrong and SNS sends email alerts so issues are caught early.

1.4 Assumptions

- Region: us-east-1 / N. Virginia
- Instance type: t3.micro - this is fine for a demo/assignment workload. For a real production app with more traffic, a larger instance type would be more appropriate.
- EC2 instances are in public subnets so they can be reached through the Internet Gateway. AWS Academy Learner Lab does not support NAT Gateways, so this was the only option available.
- Two security groups are used : one for EC2 and one for RDS. The RDS group only accepts connections from the EC2 group, not from 0.0.0.0/0.
- LabRole is used as the Elastic Beanstalk service role and LabInstanceProfile is used as the EC2 instance profile. Custom IAM roles cannot be created in AWS Academy Learner Lab so these pre-existing roles are required.
- RDS is running MySQL 8.0 on a db.t3.micro instance with 20 GiB of gp2 storage and Multi-AZ turned on.
- The Custom AMI was created from Amazon Linux 2 with PHP and the MySQLi extension installed, plus the FreshBasket app files already in place.
- The app is built in PHP (index.php) and deployed to Elastic Beanstalk as a ZIP file. PHP was chosen because it was used in the Week 4 lab and is natively supported by Elastic Beanstalk.

1.5 Complete List of AWS Services Used

- Amazon Elastic Beanstalk : deploys and manages the app environment
- Amazon EC2 (t3.micro) : runs the application instances

- Custom AMI : pre-built image used to launch EC2 instances
- EC2 Auto Scaling Group : keeps 2–8 instances running based on CPU load
- Application Load Balancer: routes incoming traffic to healthy EC2 instances
- Amazon RDS MySQL 8.0 Multi-AZ: stores order data with automatic failover
- Amazon VPC: custom isolated network for all resources
- Public Subnets (×2): one in us-east-1a, one in us-east-1b
- Internet Gateway: allows internet traffic into the VPC
- Route Table : routes 0.0.0.0/0 traffic to the Internet Gateway
- EC2 Security Group : allows HTTP port 80 and SSH port 22
- RDS Security Group: allows MySQL port 3306 from EC2 security group only
- Custom Key Pair : used for SSH access to EC2 instances
- Amazon CloudWatch: monitors CPU and triggers ASG scaling alarms
- Amazon SNS: sends email notifications for Beanstalk events
- AWS IAM (LabRole / LabInstanceProfile): provides permissions for the environment

PART 2 - AWS IMPLEMENTATION EVIDENCE

The screenshots below show all the components that were built and configured in AWS Academy Learner Lab. The screenshots below show all the components that were built and configured in AWS Academy Learner Lab. Note: The RDS security group was configured as required, but RDS instance creation was blocked by AWS Academy Learner Lab permissions with an explicit deny on `rds:CreateDBInstance`. This is documented in Screenshot 6.

Screenshot 1 - Custom VPC

This shows the custom VPC (FreshBasket-VPC) with a CIDR block of 10.0.0.0/16. Two public subnets were created in different Availability Zones, one in us-east-1a and one in us-east-1b. The route table shows a default route pointing to the Internet Gateway so instances in both subnets can reach the internet.

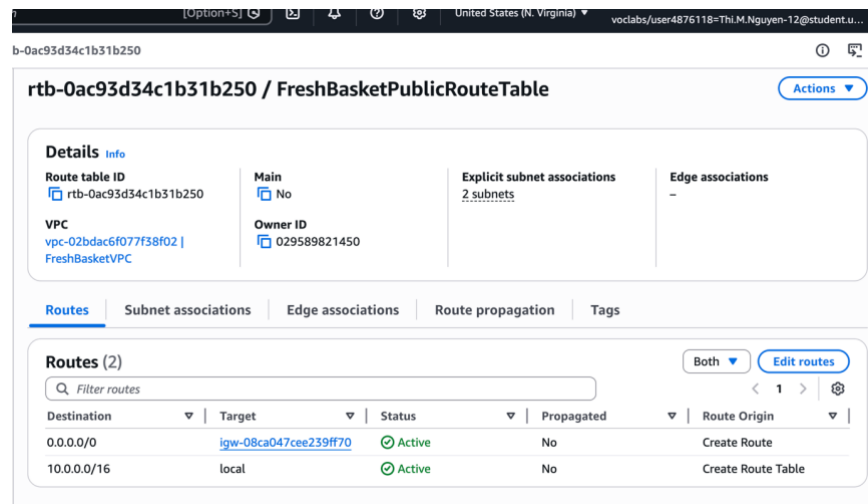


Figure 2.1 Custom VPC with two public subnets in different AZs and route table showing default route to Internet Gateway

Screenshot 2 - EC2 Security Group

The EC2 security group has two inbound rules - HTTP on port 80 to allow web traffic and SSH on port 22 for admin access. Both rules were added manually. All outbound traffic is allowed by default.

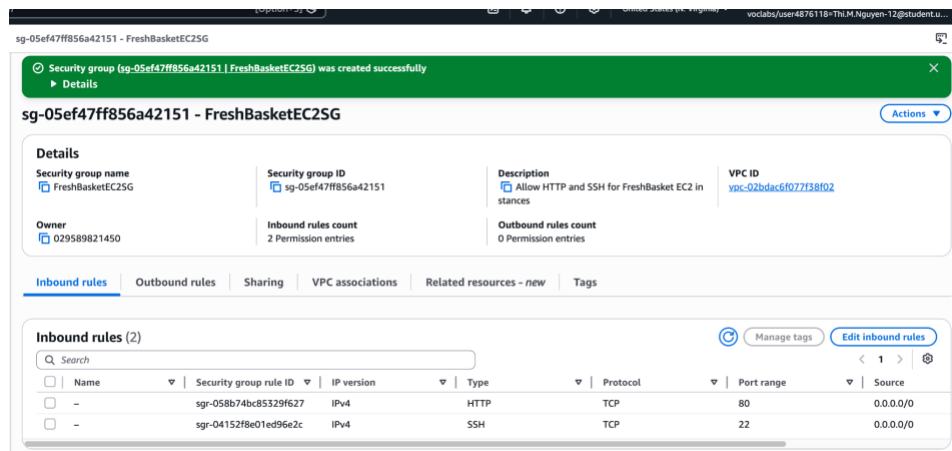


Figure 2.2 EC2 security group inbound rules showing HTTP port 80 and SSH port 22

Screenshot 3 - RDS Security Group

The RDS security group only allows MySQL traffic on port 3306. The source is set to the EC2 security group ID (not 0.0.0.0/0), which means the database can only be reached by the EC2 app instances. No direct external access is possible.

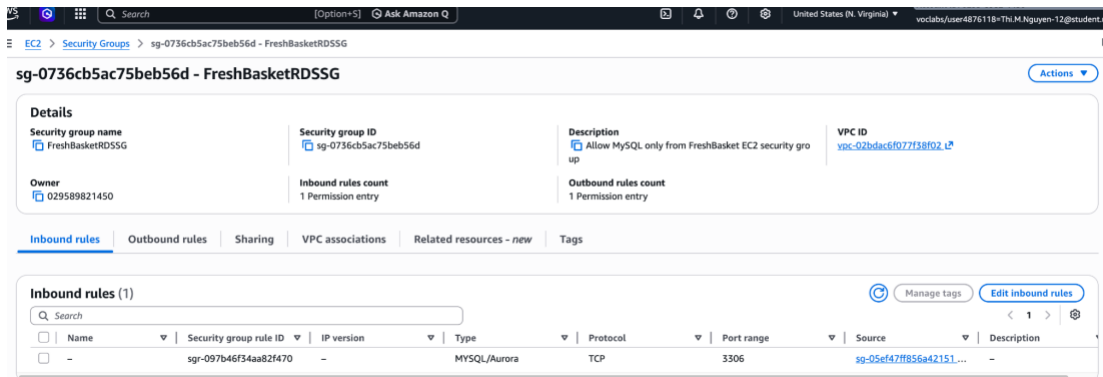


Figure 2.3 RDS security group with MySQL port 3306 restricted to EC2 security group ID as source

Screenshot 4 - Custom key pair

This key pair uses RSA and is used to securely access the EC2 instances launched for the FreshBasket environment. Using a custom key pair meets the assignment requirement and allows SSH access for administration and troubleshooting.

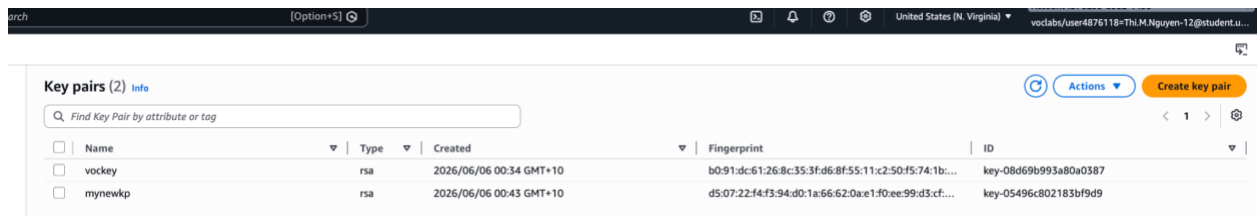


Figure 2.4 Custom key pair showing mynewkp created in EC2 Key Pairs with RSA type and key pair ID key-05496c802183bf9d9.

Screenshot 5 - Custom AMI

This AMI was created from a configured EC2 instance and can be reused to launch new application instances with the same setup. Using a custom AMI supports faster scaling because new EC2 instances can start from a pre-configured image instead of being manually configured each time.

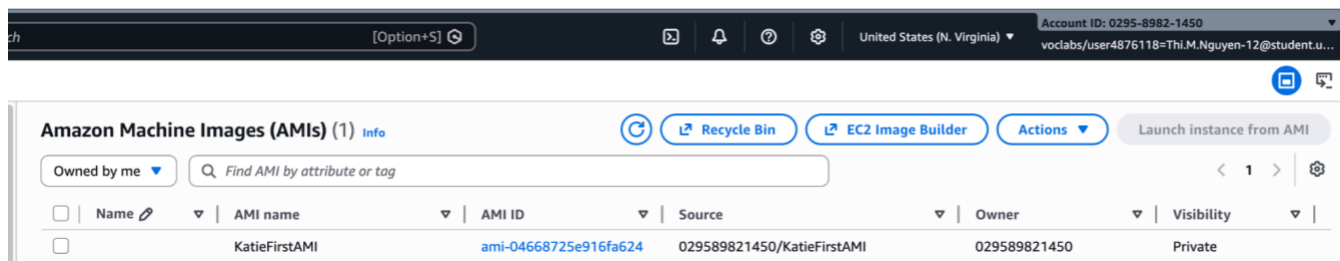


Figure 2.5 Custom AMI showing KatieFirstAMI created under “Owned by me” with AMI ID ami-04668725e916fa624

Screenshot 6 - RDS Instance Creation Error

The RDS security group was configured as required (Screenshot 3), but creating the RDS instance was blocked by AWS Academy Learner Lab permissions. The error message states that the user is not authorised to perform `rds:CreateDBInstance` due to an explicit deny in the identity-based policy `arn:aws:iam::029589821450:policy/Pvoclabs2`. This is a known restriction outside of my control

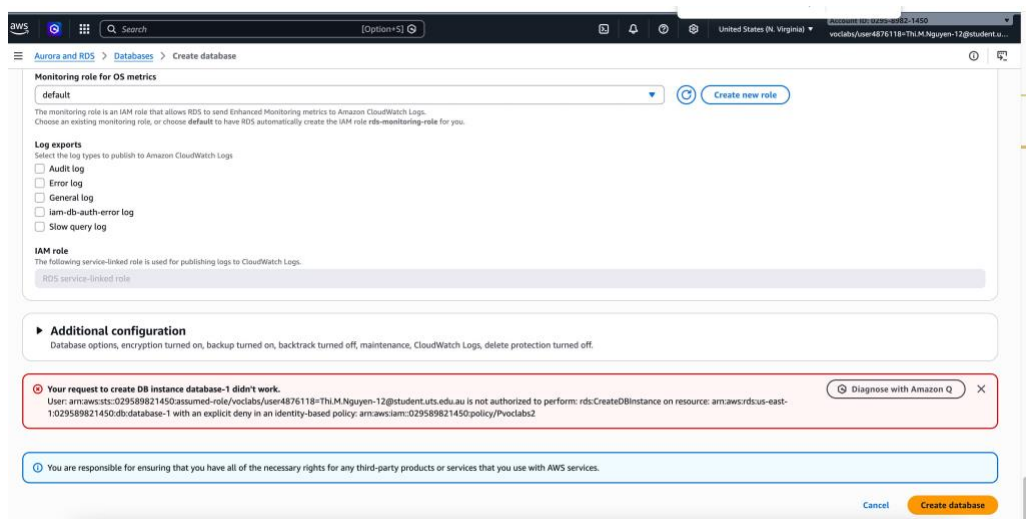


Figure 2.6 AWS error message showing `rds:CreateDBInstance` blocked by AWS Academy Learner Lab policy with an explicit deny

Screenshot 7 - Elastic Beanstalk Environment Dashboard

This shows the Elastic Beanstalk environment (FreshBasketApp-env) running in a healthy state. You can see the environment URL for the app, the health status, and the configuration confirming that LabRole is set as the service role and LabInstanceProfile is set as the EC2 instance profile. These are required for AWS Academy Learner Lab since custom IAM roles cannot be created.

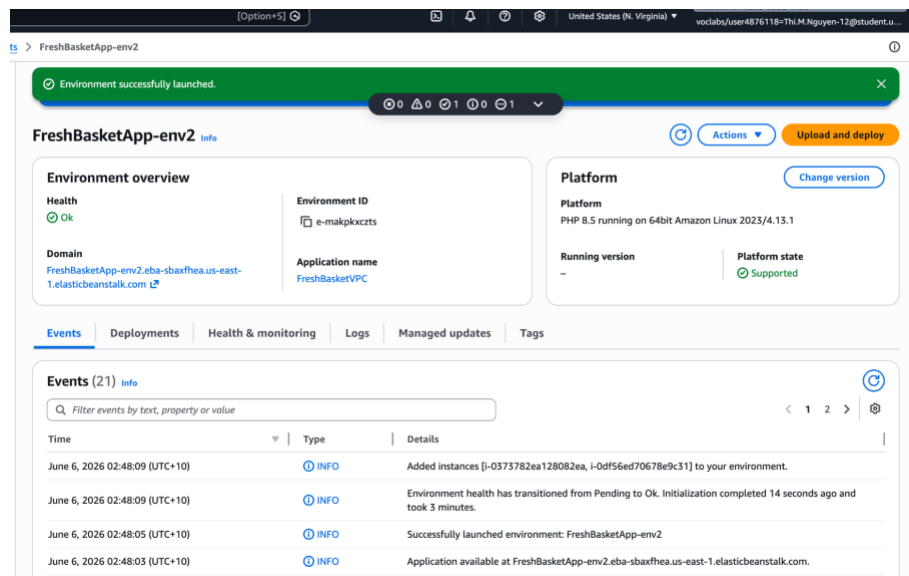


Figure 2.6 Elastic Beanstalk dashboard showing healthy status, environment URL, LabRole and LabInstanceProfile

The event log below shows Beanstalk successfully created the environment including the load balancer, Auto Scaling Group, scaling policies, CloudWatch alarms, SNS topic and two EC2 instances

Screenshot 8 - Auto Scaling Configuration

The Auto Scaling configuration shows the minimum is set to 2 instances and the maximum to 8. The CPU scaling rules are set to scale out when CPU goes above 60% and scale in when it drops below 30%. The environment is also using both public subnets across the two Availability Zones.

Configuration
Cancel
Review changes
Apply changes

Service access
Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely log in to your EC2 instances.

Service role
arn:aws:iam::52958821453:role/LabRole
EC2 key pair
mynewkey
EC2 instance profile
LabInstanceProfile

Networking and database
Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that's integrated with your environment.

Network
VPC
vpc-02bdc6027738f52
Instance subnets
subnet-01545b4b7aa776c4d,subnet-0a155045796373bd

Instance traffic and scaling
Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances
Instances
Disabled
EC2 Security Groups
sg-05af478938a421313,sg-0bba76ec355af5d8a

Capacity
Environment type
Load balanced
Min instances
2
Max instances
8

Fleet composition
On-demand instances
On-demand base
0
On-demand above base
70

Capacity rebalancing
Disabled
Scaling cooldown
360
Processor type
c4i8g4

Instance types
t3.micro, t3.small
AMI ID
ami-0a435a151a9c02b4a
Availability Zones
Any

Metric
CPUUtilization
Statistic
Average
Unit
Percent

Period
5
Breath duration
5
Upper threshold
60

Scale up increment
1
Lower threshold
30
Scale down increment
-1

Load balancer
Load balancer visibility
public
Load balancer subnets
subnet-01545b4b7aa776c4d,subnet-0a155045796373bd
Load balancer type
application

Load balancer is shared
false
Store logs
Disabled
IP address type
ipv4

Updates, monitoring, and logging
Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring
System
enhanced
Cloudwatch custom metrics - instance

Cloudwatch custom metrics - environment

Log streaming
Disabled
Retention
7
Lifecycle
false

Updates
Managed updates
Enabled
Deployment batch size
100
Deployment batch size type
Percentage

Command timeout
600
Deployment policy
ASGOnly
Health threshold
OK

Ignore health check
false
Instance replacement
false
Notifications email
katie.morgayn@gmail.com

Platform software
Lifecycle
false
Log streaming
Disabled
Allow URL, fopen
On

Display errors
Off
Document root
-
Max execution time
60

Memory limit
256M
Zip output compression
Off
Proxy server
nginx

Logs retention
7
Status logs
Disabled
Update level
minor

Environment properties
Source
Key
Value

Figure 2.7 Auto Scaling configuration showing min 2, max 8 instances and CPU thresholds at 60% and 30%

Screenshot 9 - Application Running End-to-End

This screenshot shows the FreshBasket Order Form open in a browser using the Beanstalk URL. An order was submitted with a name and produce item. The page then displays the order pulled back from the RDS MySQL database, which confirms the full flow is working, browser to Load Balancer to EC2 to RDS and back.

Events (21) info			
Filter events by text, property or value			
Time	Type	Details	
June 6, 2026 02:48:09 (UTC+10)	INFO	Added instances [i-0373782ea128082ea,i-0df56ed70678e9c31] to your environment.	
June 6, 2026 02:48:09 (UTC+10)	INFO	Environment health has transitioned from Pending to Ok. Initialization completed 14 seconds ago and took 3 minutes.	
June 6, 2026 02:48:05 (UTC+10)	INFO	Successfully launched environment: FreshBasketApp-env2	
June 6, 2026 02:48:03 (UTC+10)	INFO	Application available at FreshBasketApp-env2.eba-sbaxfhea.us-east-1.elasticbeanstalk.com.	
June 6, 2026 02:47:50 (UTC+10)	INFO	Instance deployment completed successfully.	
June 6, 2026 02:47:44 (UTC+10)	INFO	Instance deployment: You didn't include a 'composer.json' file in your source bundle. The deployment didn't install Composer dependencies.	
June 6, 2026 02:47:25 (UTC+10)	INFO	Created Load Balancer listener named: arn:aws:elasticloadbalancing:us-east-1:029589821450:listener/app/awseb-yvGyX08F0HhJ/c1811a745f29c846/awseb-yvGyX08F0HhJ/c1811a745f29c846/adc6399860bbf686	
June 6, 2026 02:47:23 (UTC+10)	INFO	Created load balancer named: arn:aws:elasticloadbalancing:us-east-1:029589821450:loadbalancer/app/awseb-yvGyX08F0HhJ/c1811a745f29c846	
June 6, 2026 02:45:50 (UTC+10)	INFO	Created CloudWatch alarm named: awseb-e-makpcczts-stack-AWSEBCloudwatchAlarmLow-sfzvuQ8fP2w	
June 6, 2026 02:45:50 (UTC+10)	INFO	Created CloudWatch alarm named: awseb-e-makpcczts-stack-AWSEBCloudwatchAlarmHigh-eaEaPvOehO4	
June 6, 2026 02:45:35 (UTC+10)	INFO	Created Auto Scaling group policy named: arn:aws:autoscaling:us-east-1:029589821450:scalingPolicy:970c3c0e-4532-4a59-bcad-699fb0dc5689:autoScalingGroupName/awseb-e-makpcczts-stack-AWSEBAutoScalingGroup-YyoZ836bHbY/policyName/awseb-e-makpcczts-stack-AWSEBAutoScalingScaleDownPolicy-B79yAHjvVog	
June 6, 2026 02:45:35 (UTC+10)	INFO	Created Auto Scaling group policy named: arn:aws:autoscaling:us-east-1:029589821450:scalingPolicy:1bb1788f-365b-4dfe-acd6-c1c3d760237a:autoScalingGroupName/awseb-e-makpcczts-stack-AWSEBAutoScalingGroup-YyoZ836bHbY/policyName/awseb-e-makpcczts-stack-AWSEBAutoScalingScaleUpPolicy-H9JHFVZobert	
June 6, 2026 02:45:35 (UTC+10)	INFO	Waiting for EC2 instances to launch. This may take a few minutes.	
June 6, 2026 02:45:35 (UTC+10)	INFO	Created Auto Scaling group named: awseb-e-makpcczts-stack-AWSEBAutoScalingGroup-YyoZ836bHbY	
June 6, 2026 02:45:09 (UTC+10)	INFO	Environment health has transitioned to Pending. Initialization in progress (running for 37 seconds). There are no instances.	
June 6, 2026 02:45:04 (UTC+10)	INFO	Created security group named: sg-0bba76ec553af3d6a	
June 6, 2026 02:45:04 (UTC+10)	INFO	Created target group named: arn:aws:elasticloadbalancing:us-east-1:029589821450:targetgroup/awseb-AWSEB-7QFOEVHFDZEK/14369fcff560a4d	
June 6, 2026 02:44:49 (UTC+10)	INFO	Created security group named: sg-02539c3951015048	
June 6, 2026 02:44:31 (UTC+10)	INFO	Created SNS Notification Topic. ARN: arn:aws:sns:us-east-1:029589821450:ElasticBeanstalkNotifications-Environment-FreshBasketApp-env2	
June 6, 2026 02:44:27 (UTC+10)	INFO	Using elasticbeanstalk-us-east-1-029589821450 as Amazon S3 storage bucket for environment data.	

Figure 2.8 FreshBasket app running in browser via Beanstalk URL, showing successful read/write from RDS MySQL

Screenshot 10 - Email Notifications

AWS Elastic Beanstalk Notification - Environment health has transitioned from Ok to Degraded. No ... [Summarise](#) [🔍](#) [🔄](#) [📧](#)

AN [AWS Notifications <no-reply@sns.amazonaws.com>](#) Today at 4:41

To: [Thi Minh Thu Nguyen](#)

[You don't often get email from no-reply@sns.amazonaws.com. Learn why this is important at <https://aka.ms/LearnAboutSenderIdentification>]

Timestamp: Fri Jun 05 18:41:09 UTC 2026
 Message: Environment health has transitioned from Ok to Degraded. No data received from 2 out of 3 instances. 66.7 % of the requests to the ELB are erroring with HTTP 4xx. Insufficient request rate (1.5 requests/min) to determine application health (6 minutes ago).

Environment: FreshBasketApp-env2
 Application: FreshBasketVPC

Environment URL: <http://FreshBasketApp-env2.eba-sbaxfhea.us-east-1.elasticbeanstalk.com>
 NotificationProcessId: 4d272ee1-4ae1-4124-9798-516bd4eedc0

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:029589821450:ElasticBeanstalkNotifications-Environment-FreshBasketApp-env2:415572f2-b200-4684-aca7-428addb33541&Endpoint=Thi.M.Nguyen-12@student.uts.edu.au>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

Figure 2.9 Beanstalk event notification email showing subject line and timestamp

aws Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:
 arn:aws:sns:us-east-1:029589821450:ElasticBeanstalkNotifications-Environment-FreshBasketApp-env2:415572f2-b200-4684-aca7-428addb33541

If it was not your intention to subscribe, [click here to unsubscribe](#).

Figure 2.10 SNS subscription confirmation email showing the notification endpoint is active

References

University of Technology Sydney. (2026). *Week 3 lab: Infrastructure as a service - provisioning and managing virtual machines on AWS* [Lecture slides]. Canvas, 41001/42904 Cloud Computing and Software as a Service, Autumn 2026.

University of Technology Sydney. (2026). *Week 4 lab: Platform as a service - deploying and managing applications with AWS Elastic Beanstalk* [Lecture slides]. Canvas, 41001/42904 Cloud Computing and Software as a Service, Autumn 2026.